Array Algorithms

Searching Algorithms:

Linear Search: Brute Force Algorithm (also known as Exhaustive Search)

Linear search is a simple searching algorithm that checks each element in the array sequentially until the target element is found or the end of the array is reached.

Time Complexity: **Best Case: O (1), Worst Case: O (N), Average Case: O (N)**

```
import java.util.Scanner;
class LinearSearch {
    public static void main(String args[]) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int a[] = new int[n];
        for(int i = 0; i < a.length; i++) {
            a[i] = sc.nextInt();
        }
        int s = sc.nextInt();
        boolean found = false;
        for(int i = 0; i < a.length; i++) {
            if(a[i] == s) {
                System.out.println("Element found at index " + i);
                found = true;
                break;
            }
        }
        if(!found) {
            System.out.println("Element not found");
        }
    }
}
```

Binary Search: (Divide and Conquer Algorithm)

Definition:
Binary search is a searching algorithm that finds the position of a target value within a sorted array by repeatedly dividing the search interval in half. Array must be sorted in this case.

Time Complexity: **Best Case: O (1), Worst Case: O (Log N), Average Case: O (Log N)**

```
import java.util.Scanner;
public class Main {
    public static void main(String args[]) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int a[] = new int[n];
        for(int i = 0; i < a.length; i++) {
            a[i] = sc.nextInt();
        }
        int s = sc.nextInt();
        int f = 0, l = a.length - 1;
        while(f <= l) {
            int mid = (f + l) / 2;
            if(a[mid] == s) {
                System.out.println("Element found at index " + mid);
                break;
            } else if(a[mid] < s) {
                f = mid + 1;
            } else {
                l = mid - 1;
            }
        }
        if(f > l) {
            System.out.println("Element not found");
        }
    }
}
```

Sorting Algorithms:

Selection Sort:

Selection Sort is a simple comparison-based sorting algorithm. It repeatedly selects the minimum element from the unsorted portion of the array and swaps it with the first unsorted element. This process is repeated until the entire array is sorted.

Time Complexity: **Best Case: O (N^2), Worst Case: O (N^2), Average Case: O (N^2)**

Selection Sort for sorting in ascending order:

```
import java.util.Scanner;
public class SelectionSort {
    public static void main(String args[]) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int a[] = new int[n];
        for(int i = 0; i < a.length; i++) {
            a[i] = sc.nextInt();
        }
        for(int i = 0; i < a.length - 1; i++) {
            int m = i;
            for(int j = i + 1; j < a.length; j++) {
                if(a[j] < a[m]) {
                    m = j;
                }
            }
            int temp = a[i];
            a[i] = a[m];
            a[m] = temp;
        }
        for(int i = 0; i < a.length; i++) {
            System.out.print(a[i] + " ");
        }
    }
}
```

Bubble Sort:

Bubble Sort is a simple comparison-based sorting algorithm. It repeatedly steps through the list, compares adjacent elements, and swaps them if they are in the wrong order. This process is repeated until no swaps are needed, indicating that the list is sorted.

Time Complexity: **Best Case: O (N^2), Worst Case: O (N^2), Average Case: O (N^2)**

If Optimized: Best Case: O (N), Worst Case: O (N^2), Average Case: O (N^2)

Bubble sort for sorting in ascending order:

```
import java.util.Scanner;
public class BubbleSort {
    public static void main(String args[]) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int a[] = new int[n];
        for(int i = 0; i < a.length; i++) {
            a[i] = sc.nextInt();
        }

        for(int i = 0; i < a.length - 1; i++) {
            for(int j = 0; j < a.length - 1 - i; j++) {
                if(a[j+1] < a[j]){
                    int temp = a[j];
                    a[j] = a[j + 1];
                    a[j + 1] = temp;
                }
            }
        }

        for(int i = 0; i < a.length; i++) {
            System.out.print(a[i] + " ");
        }
    }
}
```

Duplicate Removal

```
import java.util.Scanner;
public class DuplicateRemoval {
    public static void main(String args[]) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int a[] = new int[n];
        for(int i = 0; i < a.length; i++) {
            a[i] = sc.nextInt();
        }
        int b[]=new int[n];
        int k=0;

        for(int i = 0; i < a.length; i++) {
            boolean found=false;
            for(int j=0; j<b.length; j++){
                if(a[i]==b[j]){
                    found=true;
                    break;
                }
            }
            if(!found){
                b[k++]=a[i];
            }
        }
        System.out.println("After removing duplicates");
        for (int i = 0; i < k; i++) {
            System.out.print(b[i]);
        }
    }
}
```

Duplicate Collection

```
import java.util.Scanner;
public class DuplicateCollection {
    public static void main(String args[]) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int a[] = new int[n];
        for(int i = 0; i < a.length; i++) {
            a[i] = sc.nextInt();
        }
        int b[]=new int[n];
        int k=0;

        for(int i = 0; i < a.length; i++) {
            int c=0;
            for(int j=0; j<a.length; j++){
                if(a[i]==a[j])c++;
            }
            if(c>1){
                boolean found=false;
                for(int j=0; j<b.length; j++){
                    if(a[i]==b[j]){
                        found=true;
                        break;
                    }
                }
                if(!found){
                    b[k++]=a[i];
                }
            }
        }
```

```
        System.out.println("Duplicates=");
        for (int i = 0; i < k; i++) {
            System.out.print(b[i]);
        }
    }
}
```

Array Reversal Without Second Array

```
import java.util.Scanner;
public class ArrayReversal {
    public static void main(String args[]) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int a[] = new int[n];
        for (int i = 0; i < a.length; i++) {
            a[i] = sc.nextInt();
        }
        int f = 0, l = a.length - 1;
        while (f < l) {
            int k = a[f];
            a[f] = a[l];
            a[l] = k;
            f++;
            l--;
        }
        System.out.println("Reversed array:");
        for (int i = 0; i < a.length; i++) {
            System.out.print(a[i] + " ");
        }
     }
}
```

Comparison Logic Equivalence

Primitives (including char)	String (Alphabetical/ Lexicographical)
a<b	a.compareTo(b)<0
a>b	a.compareTo(b)>0
a==b	a.compareTo(b)==0 OR a.equals(b)

Subhasis